

Dual-Issue Execution of Mixed Integer and Floating-Point Workloads on Energy-Efficient In-Order RISC-V Cores

Luca Colagrande
Integrated Systems Laboratory (IIS)
ETH Zurich
Zurich, Switzerland
colluca@iis.ee.ethz.ch 

Luca Benini
Integrated Systems Laboratory (IIS)
ETH Zurich
Zurich, Switzerland
lbenini@iis.ee.ethz.ch 

Abstract—To meet the computational requirements of modern workloads under tight energy constraints, general-purpose accelerator architectures have to integrate an ever-increasing number of extremely area- and energy-efficient processing elements (PEs). In this context, single-issue in-order cores are commonplace, but lean dual-issue cores could boost PE IPC, especially for the common case of mixed integer and floating-point workloads. We develop the COPIFT methodology and RISC-V ISA extensions to enable low-cost and flexible dual-issue execution of mixed integer and floating-point instruction sequences. On such kernels, our methodology achieves speedups of 1.47x, reaching a peak 1.75 instructions per cycle, and 1.37x energy improvements on average, over optimized RV32G baselines.

Index Terms—RISC-V, dual-issue, energy efficiency, general purpose

I. INTRODUCTION

With the end of Dennard scaling, energy-efficiency has become a primary constraint in the design of high-performance computer architectures [1]. Simultaneously, the rapid growth of Machine Learning (ML) models, requiring immense computational resources for training and inference, are dramatically escalating the demand for high-performance computing power [2].

To meet this challenge, modern high-performance architectures integrate an ever-increasing number of cores, designed to meet extreme area- and energy-efficiency targets to enable large-scale replication [3]. In-order single-issue cores with shallow pipelines are favoured over power- and resource-hungry deeply-pipelined out-of-order processors.

Within the context of massively-parallel general-purpose accelerators, few architectures propose cores with limited multiple-issue capabilities. Nvidia's Turing architecture, for instance, implements concurrent execution of FP32 and INT32 operations [4], allowing the Floating-Point (FP) math datapath to remain active during the execution of non-FP-math instructions. According to the authors, this results in a 36% average increase in throughput across several gaming workloads. Unfortunately, due to the proprietary nature of the architecture, implemen-

tation details and their area and power impact remain undisclosed.

A similar form of dual-issue execution was implemented in the Snitch processor [5], coupling a tiny streamlined single-issue in-order RV32I control core and a 64-bit SIMD-capable Floating Point Unit (FPU). Instructions are fetched and issued by the control core, and offloaded to the FPU as required. To support dual-issue execution, the Floating Point Subsystem (FPSS) integrates a small instruction buffer to cache small FP loop bodies, the Floating Point Repetition (FREP) buffer. FP instructions are first issued by the integer core, and cached in the FREP buffer. In the remaining loop iterations, the FREP sequencer can independently issue instructions, while the integer core is free to fetch, and potentially execute, successive instructions. Its authors refer to this scheme as *pseudo dual-issue execution*.

This approach implies the following constraints:

- the FP loop body must be small enough to fit entirely in the FREP buffer;
- the programmer or compiler need to explicitly mark the FP loop for FREP execution in software;
- there can be no dependencies between the FP and integer threads.

The latter constraint follows from the asynchronous execution of the two threads, between which no ordering guarantees are enforced. While this enables Snitch's dual-issue capabilities at a very low power and area cost, it limits flexibility. In particular, it hinders the execution of computations featuring a mix of interdependent integer and FP instructions. We refer to these as mixed integer and FP instruction sequences.

In this paper, we present *COPIFT*, a generic method to develop **Co-Operative Parallel Integer and Floating-point Threads** in Snitch, overcoming the limitation imposed by pseudo dual-issue, and enabling low-cost dual-issue execution of mixed integer and FP instruction sequences.

To summarize our contributions, we:

- Present COPIFT, a generic methodology to enable sustained dual-issue execution of mixed integer and

FP instruction sequences in Snitch, a state-of-the-art open-source RISC-V processor.

- Develop a set of Instruction Set Architecture (ISA) extensions to broaden the range of applications suited for pseudo dual-issue execution, in conjunction with the COPIFT method.
- Implement COPIFT-accelerated codes for selected mixed integer and FP instruction sequences, achieving average speedups of $1.47\times$ over optimized RV32G baselines, and a peak IPC of 1.75.
- Present a first account of the energy-efficiency of pseudo multiple-issue execution schemes, demonstrating average energy reductions of $1.37\times$ with COPIFT, over optimized RV32G baselines.

Our implementation is fully open source and performance experiments are reproducible using free software.¹

II. IMPLEMENTATION

A. COPIFT Methodology

Akin to other ISAs, RISC-V defines distinct Register Files (RFs) for the RV32I base integer ISA and the “D” standard extension [6]. As a result, integer and FP instructions operate mostly on independent sets of registers. This fundamental characteristic allows us to define two threads with independent state, comprising respectively the integer RF and RV32I instructions and the FP RF and “D” extension instructions. Several notable FP instructions, however, also require access to the integer RF, consuming operands from integer source registers (FP load, store, conversion and move instructions) or producing results into integer destination registers (FP comparisons). Such instructions conflict with the independent thread abstraction, precluding pseudo-dual issue execution.

Partly relaxing this constraint, Snitch implements Stream Semantic Registers (SSRs) [7], a custom ISA extension to stream data from memory directly into the RF, eliminating explicit load and store instructions. SSRs support any *affine stream* or memory access pattern which can be described as an affine function of the induction variables of a four-dimensional loop nest. Extending the SSR mechanism, Indirection Stream Semantic Registers (ISSRs) [8], further provide support for *indirect streams* or arbitrary memory access patterns, provided a list of relative addresses encoding the pattern. SSRs enable memory accesses without introducing dependencies on the integer RF, solving only a particular instance of the problem. Conversely, our methodology tackles any form of interdependency between integer and FP threads.

Consider the code in Figure 1a, computing the exponential of an array of single-precision FP numbers. Compiling this code for the RV32G architecture results in the mix of integer and FP instructions shown in Figure 1b. Dependencies between these instructions are marked with

red and blue boxes in the consuming and producing instructions, respectively. These can be classified into:

- Type 1** Dynamic memory dependencies, from FP load-stores with dynamically-computed addresses.
- Type 2** Static memory dependencies, from FP load-stores with statically-determined addresses.

In the most general case, a third type may be present:

- Type 3** Register dependencies, from FP conversion, move and comparison instructions.

Due to these dependencies, we cannot trivially partition integer and FP instructions for pseudo dual-issue execution. The COPIFT method addresses the problem by eliminating or increasing the distance between dependent instructions. It involves the following steps:

- Step 1** Construct a Data Flow Graph (DFG) from the RISC-V assembly and identify all dependencies between integer and FP instructions.
- Step 2** Partition the DFG on the identified edges, such that there exists an acyclic precedence-relation among the subgraphs, and the number of edges between them is minimized.
- Step 3** Reorder the instructions according to the subgraph ordering established in Step 2.
- Step 4** Apply loop tiling to block the data, and loop fission to reschedule the subgraph computations on each block.
- Step 5** Apply software pipelining and multiple buffering to overlap subgraph computations.
- Step 6** Map all FP load-store instructions to SSRs.
- Step 7** Map all FP loops to FREP loops, and reorder them to precede the integer loop.

To clarify each step, we walk through how to apply COPIFT to the exponential kernel in Figure 1a.

Figure 1c shows the DFG of the basic block in Figure 1b. For brevity, we omitted instructions 24 and 25 and associated dependencies, as we will see that these can be simplified in Step 6. Following Step 2, we cut edges $4 \rightarrow 5$, $12 \rightarrow 18$ and $14 \rightarrow 18$. This alone results in two subgraphs for which a precedence-relation cannot be established. We therefore further partition the graph, cutting edge $21 \rightarrow 22$. We obtain three subgraphs, each defining a phase of the computation with clear ordering requirements w.r.t. the others: FP Phase 0 must precede integer Phase 1, which in turn must precede FP Phase 2. This allows us to reorder the instructions in consecutive groups of integer-only or FP-only instructions, while respecting the dependencies within each loop iteration (Step 3). The resulting code would resemble Figure 1d, where each function would be replaced by the respective inline assembly code.

In Step 4, we then apply loop tiling [9] to obtain the code in Figure 1e. Each phase is now applied to an entire block of inputs, before passing on a block of intermediate results to the next phase. Implicitly, this step requires allocating block-sized buffers for each variable associated to an edge between subgraphs. Additionally,

¹https://github.com/colluca/snitch_cluster/tree/copift

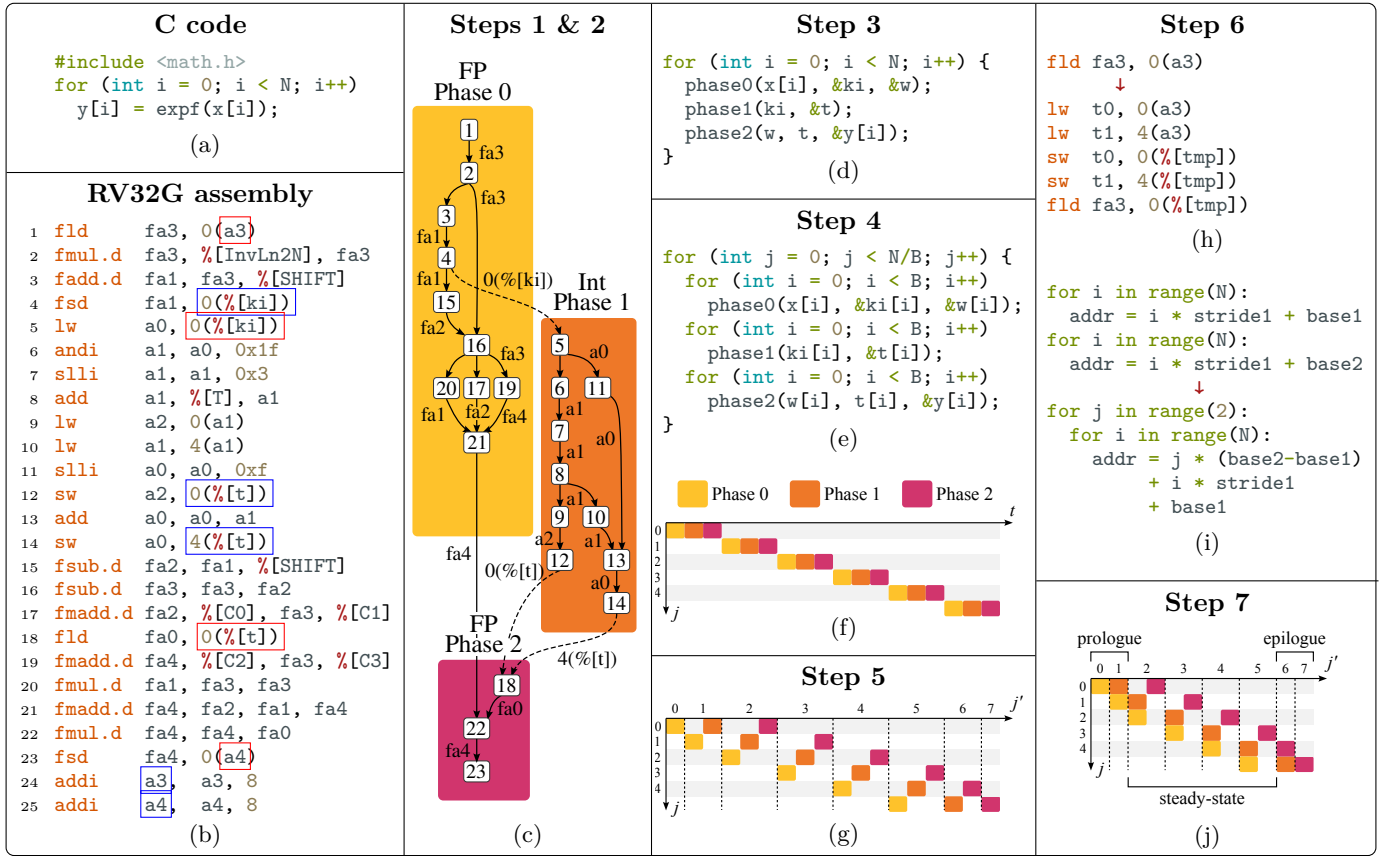


Fig. 1: Illustration of the COPIFT methodology applied to a kernel calculating the exponential of a vector.

all inter-phase communication originally through the RF, must be preventively spilled to memory. To reduce spill communication overheads and memory requirements, it is important to minimize the number of edges between subgraphs in Step 2.

Step 5 prepares for the parallel execution of the FP and integer phases, by applying software pipelining [10] to increase the distance between dependent phases. Figures 1f and 1g illustrate the original and modified block schedules, respectively. Within each j' iteration in the new schedule, every phase operates on different blocks of data j , allowing them to be parallelized in Step 7. Implicitly, this requires replicating the buffers, as e.g. Phase 0 produces different blocks of w before the first is consumed by Phase 2. The exact number of replicas for each buffer equals the distance between the subgraphs connected by the respective edge in the total ordering established in Step 2, plus one. In our example, the w buffer, associated to the edge between Phase 0 and 2, must be replicated three times.

Steps 6 to 7 enable the integer and FP phases within each block iteration to be executed in parallel. In Step 4, we spilled all inter-phase communication through the RFs to memory, introducing additional load-store instructions. In Step 6, we eliminate all FP load-stores by mapping the respective memory accesses to SSRs. The accesses

performed by the two FP phases in our example can be described as a combination of 6 one-dimensional streams: reads from blocks of x , w and t , and writes to blocks of w , ki and y . All streams originate from tiling in Step 4, and can thus be naturally represented as regular accesses into contiguous arrays. Special treatment is required to convert streams of addresses originating from Type 1 dependencies, into regular streams of data for the FP thread. This can be done in software by converting Type 1 to Type 2 dependencies as shown in Figure 1h. This involves prefetching the data in the integer thread and storing it in a regular layout for the SSRs. Alternatively, Type 1 dependencies can be directly mapped to ISSRs, performing the indirection on the regular stream of addresses in hardware. Finally, we note that in cases where the Type 1 address stream can be statically generated by an affine function of up to four variables, the entire address calculation can be eliminated and performed by an SSR, which is the case for the x and y streams in our example. The associated dependency and all artifacts produced by our methodology can be optimized away, which allowed us to ignore instructions 24 and 25 from the start.

To map the 6 streams to the 3 SSRs available in every Snitch core, we apply a *stream fusion* technique, which involves mapping multiple lower-dimensional affine streams

Kernel	Baseline			Step 4		Steps 5 to 6		Max Block	COPIFT		I'	S''	$\nabla S'$
	#Int	#FP	TI	Int Ld/St	#Buff.	FP Ld/St	#Buff.		#Int	#FP			
<code>expf</code>	43	52	0.83	0	5	-4	13	157	43	36	1.84	1.83	2.21
<code>logf*</code>	39	52	0.75	+4	6	-4	12	273	57	36	1.63	1.75	1.6
<code>poly_lcg*[†]</code>	44	80	0.55	+3	3	0	6	341	72	80	1.9	1.55	1.55
<code>pi_lcg*[†]</code>	44	56	0.79	+3	3	0	6	341	72	56	1.78	1.79	1.39
<code>poly_xoshiro128p*[†]</code>	172	80	0.47	+3	3	0	6	341	200	80	1.4	1.47	1.26
<code>pi_xoshiro128p*[†]</code>	172	56	0.33	+3	3	0	6	341	200	56	1.28	1.33	1.14

Require support for `*fcvt.d.w` and `†flt.d`.

TABLE I: Characteristics of the evaluated kernels, ordered by expected speedup S' .

to a single higher-dimensional affine stream. Figure 1i illustrates the process of merging two one-dimensional affine streams. Using this technique, we merge the x and t read streams and the w , ki and y write streams.

Finally, FP block computations are mapped to FREP loops (Step 7). Since instructions for the first iteration of an FREP loop are issued by the integer core, the FREP loops must precede the integer loop in program order, such that remaining iterations overlap with the integer thread. Similarly, for the integer thread to overlap with both FP Phase 0 and 2, these should be fused to a single FREP loop in block iterations where both phases are performed. The resulting schedule is shown in Figure 1j.

The steps in this methodology can be followed by developers to obtain an optimized mixed C and assembly representation.

B. COPIFT ISA Extensions

FP conversion and comparison instructions are critical for mixed integer and FP workloads. To support these, we can alter their semantics under FREP operation, to operate entirely on the FP RF. Communication between the integer and FP RFs is spilled to memory, through the use of additional load-store instructions, decoupling it from the computation, so the target instructions can find all operands directly in the FP RF. This technique can be combined with SSRs, to implicitly encode the FP load-store operations.

Albeit not strictly necessary, we chose to develop custom instructions to carry the altered semantics, for each of the relevant “D” extension instructions, i.e. `fcvt.w[u].d`, `fcvt.d.w[u]`, `feq.d`, `flt.d`, `fle.d` and `fclass.d`. We copy the original encodings, allocating the new instructions in the `custom-1` opcode space [6].

III. RESULTS

We implement a Snitch cluster hosting one compute core in GlobalFoundries’ 12LP+ FinFET technology using Fusion Compiler, with a target clock frequency of 1 GHz. Our extensions introduce negligible area and timing overheads, within the margin of synthesis process variability.

All experiments are conducted at the target frequency in cycle-accurate RTL simulations using QuestaSim 2023.4. Switching activities are extracted from post-layout simulations, and used for power estimation in PrimeTime,

assuming typical operating conditions of 25 °C and 0.8 V supply voltage.

A. Performance Evaluation

To evaluate the benefits of the COPIFT method and ISA extensions, we implement a set of mixed integer and FP workloads with diverse characteristics, extracted from two different domains:

- Hit and miss Monte Carlo integration methods [11], evaluating two different integration problems and pseudo-random number generators. Integer instructions are typically employed in pseudo-random number generation, while the integration is performed in double-precision FP arithmetic.
- Transcendental function evaluation on a vector of single-precision FP numbers. Reference implementations are taken from the GNU C Library [12] v2.40, and involve integer instructions for bit-manipulation and double-precision FP arithmetic to evaluate approximating polynomials.

The latter in particular, applied to the exponential function, is the main component of softmax operations, which consume a considerable fraction of cycles in modern Large Language Models (LLMs) [13].

We implement Snitch-optimized RV32G-compliant baseline codes and COPIFT-accelerated variants for each kernel, compiled using a custom toolchain for Snitch based on LLVM 18.1.4 with `-O3` optimization.

Table I lists all of the evaluated kernels, with their unique characteristics. As the table shows, the number of integer and FP instructions can vary as a result of applying the COPIFT method, as e.g. integer load-stores are introduced in Step 4 and FP load-stores are eliminated in Step 6. These easily measurable kernel characteristics can be used to estimate the speedup potential of the COPIFT method. In fact, assuming similar IPCs in the integer and FP threads, the speedup S can be approximated as a function of the number of integer and FP instructions:

$$S := \frac{t_{\text{int}}^{\text{base}} + t_{\text{fp}}^{\text{base}}}{\max(t_{\text{int}}^{\text{copift}}, t_{\text{fp}}^{\text{copift}})} \approx \frac{n_{\text{int}}^{\text{base}} + n_{\text{fp}}^{\text{base}}}{\max(n_{\text{int}}^{\text{copift}}, n_{\text{fp}}^{\text{copift}})} = S' \quad (1)$$

Similarly, we can approximate the IPC improvement I :

$$I := S * \frac{n_{\text{int}}^{\text{copift}} + n_{\text{fp}}^{\text{copift}}}{n_{\text{int}}^{\text{base}} + n_{\text{fp}}^{\text{base}}} \approx \frac{n_{\text{int}}^{\text{copift}} + n_{\text{fp}}^{\text{copift}}}{\max(n_{\text{int}}^{\text{copift}}, n_{\text{fp}}^{\text{copift}})} = I' \quad (2)$$

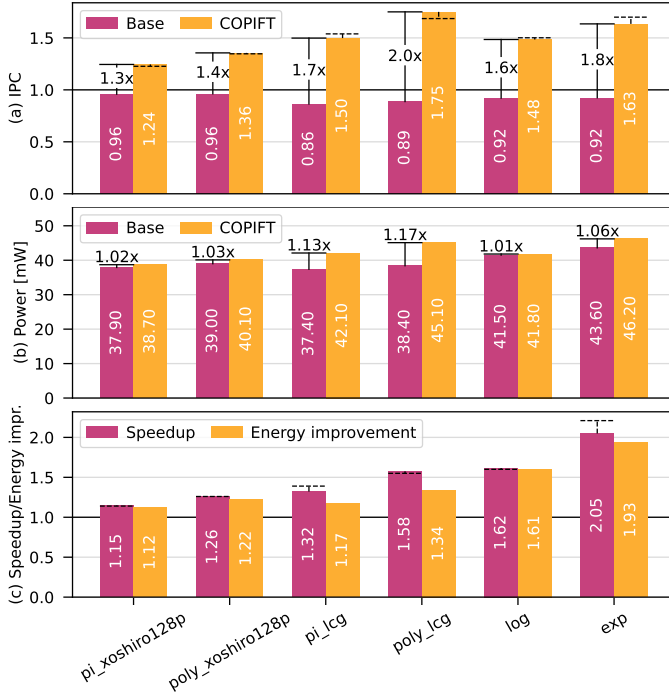


Fig. 2: Comparison of various metrics between COPIFT and baseline codes. Dashed lines indicate expected values: (a) the IPC derived from I' and (c) the speedup S' .

If the number of instructions does not differ significantly between the two code variants, we can further approximate the speedup as a function of the number of instructions in the original implementation alone:

$$S' \approx \frac{n_{\text{int}}^{\text{base}} + n_{\text{fp}}^{\text{base}}}{\max(n_{\text{int}}^{\text{base}}, n_{\text{fp}}^{\text{base}})} = 1 + \frac{\min(n_{\text{int}}^{\text{base}}, n_{\text{fp}}^{\text{base}})}{\max(n_{\text{int}}^{\text{base}}, n_{\text{fp}}^{\text{base}})} = 1 + \text{TI} = S'' = I'' \quad (3)$$

Intuitively, this shows how the speedup approximately depends on the thread imbalance (TI), as defined above.

Figure 2a compares the IPC of a steady-state iteration in the two implementations, for all kernels ordered by increasing expected speedup S' . We observe a significant geomean IPC improvement of 1.62 $\times$, and a peak IPC of 1.75, as a demonstration of dual-issue performance. As hypothesized, the IPC improvement correlates with I' . The deviations observed in the `poly_lcg` and `pi_lcg` kernels are due to the hypothesis made in Equation (1) being invalid. In fact, both benchmarks present stalls in the Pseudo-Random Number (PRN) generation with the Linear Congruential Generator (LCG), which are due to structural hazards on the register file's writeback port, and could not be eliminated by unrolling. These stalls effectively balance out the execution times of the integer and FP threads in the `poly_lcg` kernel, leading to a near-ideal IPC improvement of 1.97 $\times$, while unbalancing the

²Using equality $a + b = \max(a, b) + \min(a, b)$

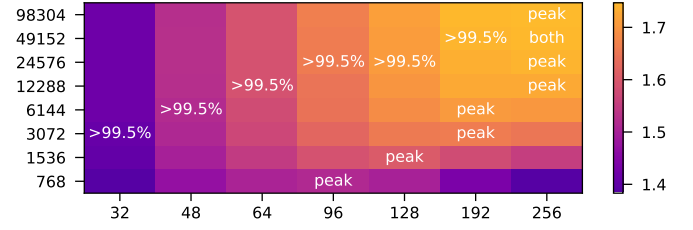


Fig. 3: IPC of the `poly_lcg` kernel for various problem and block sizes.

`pi_lcg` kernel. Finally, the `exp` kernel also shows a small deviation. This is due to the instruction overhead required to program the SSRs and switch buffers in every block iteration, which is not amortized as effectively on smaller blocks, as this kernel features.

To understand the impact of block size and problem size on performance, in Figure 3 we plot the IPC for various configurations of the `poly_lcg` kernel. As expected, IPC increases with problem size, as fixed prologue and epilogue overheads are amortized over a larger number of block iterations. Smaller blocks result in smaller prologue and epilogue overheads and a larger number of block iterations, for equal problem sizes, and can thus amortize these overheads more effectively. This trend is displayed in the ">99.5%" annotations, marking the smallest problem size which achieves >99.5% of the maximum IPC attainable with each block size. On the other hand, smaller blocks are less able to amortize overheads within the block, e.g. from SSR programming and switching buffers. As such, smaller block sizes feature lower maximum IPCs, when all other overheads are fully amortized. In this tradeoff, there exists an optimal block size per problem size, which we annotate with the "peak" labels. As the problem size increases, the peak shifts towards larger block sizes, up to the maximum size we can fit in L1 memory, reported in Table I. Finally, we note that, as we tend to amortize all overheads, the IPC converges to the steady-state IPC presented in Figure 2a. In the following discussions we will thus restrict our analysis to steady-state measurements.

Figure 2c shows how the IPC increase translates to improved kernel performance. Again, we observe a significant geomean speedup of 1.47 $\times$, with a peak 2.05 $\times$ speedup on the `exp` kernel. The deviations from the expected speedup S' are a consequence of the same overheads affecting the IPC. Finally, we note that speedups greater than two are possible, as a result of additional optimizations, such as load-store elision with the SSRs, on top of dual-issue execution.

B. Energy Evaluation

Figure 2b compares the power consumption of the two implementations. The Monte Carlo benchmarks exhibit a lower base power consumption than their counterparts, due to the inactivity of the Direct Memory Access (DMA) engine and reduced L1 memory accesses. The increase in

power is consistently correlated with the IPC increase, with exception of the `log` and `exp` kernels. In fact, after separating the FP instructions, the integer loop bodies of these two kernels comprise less than 64 instructions, which entirely fit in Snitch's L0 I\$. As a result, the I\$ consumes significantly less power in the COPIFT implementations, which do not experience thrashing at the L0 level.

Ultimately, as the power consumption is dominated by constant components such as the clock network activity, we observe a maximum power increase of only $1.17\times$, and a geomean power increase of $1.07\times$ over all kernels. As a result, COPIFT is still largely beneficial in overall energy terms, as shown in Figure 2c. The reduced execution time, resulting from dual-issue execution, outweighs the increase in power consumption, resulting in geomean energy savings of $1.37\times$ on the measured benchmarks, and a peak $1.93\times$ saving on the `exp` kernel.

IV. RELATED WORK

To issue multiple instructions in parallel while preserving program correctness, multiple-issue processors must be able to identify dependencies within an instruction stream. Multiple-issue capabilities are often built into out-of-order processors, leveraging the facilities which enable out-of-order execution, such as reservation stations, register renaming and reorder buffers, which lead to high area and power overheads. On the other hand, in-order multiple-issue processors can be implemented in two flavours: statically-scheduled superscalar processors and VLIW processors [14]. In-order superscalar processors integrate logic to identify hazards between instructions in the issue packet and those in execution, while VLIW processors offload part of the job to the compiler, in charge of packing independent instructions into bundles of a predetermined size, at the cost of increased code size.

Few recent works in the literature look into the area- and energy-efficient design of dual-issue in-order RISC processors. All of these target dual-issue execution of arbitrary instruction pairs, with minor restrictions; as a consequence, they require a minimum of four read and two write (4R2W) ports into the RF. Lozano et al. [15] design a novel latency-optimized 4R2W-port SRAM-based RF implementation for soft FPGA cores, yet their implementation incurs a $4\times$ cost in physical RAM blocks compared to the single-issue baseline, which also accounts for the majority of the $+18\%$ power increase. Similarly, Kra et al. [16] implement a dual-issue in-order RISC-V core based on CV32E40P, duplicating instruction decode, issue and execution stages and integrating a 5R3W-port RF with cross-forwarding support from all write to all read ports, for a total 60% area increase. Patsidis et al. [17] claim to further push the area and energy efficiency limits of dual-issue RISC-V cores, by restricting dual-issue to 16-bit compressed instructions, fitting within a single 32-bit instruction fetch port. They extend the RF with extra read ports, and a partitioned design to harvest the energy

benefits from locality of reference to eight commonly-addressed registers by compressed instructions.

Wygrzwalski et al. [18] present a pseudo dual-issue mechanism for small loop bodies, which resembles the Snitch implementation. A separate instruction buffer is allocated as a source of instructions for the second issue slot. Differently from Snitch, they rely on a banked RF design to prevent structural hazards on the RF ports, restricting parallel execution to pairs of instructions accessing different register banks. Conversely, Snitch targets pseudo dual-issue execution of integer and FP instructions, leveraging the separate RFs available by ISA design. Unfortunately, Wygrzwalski's design is targeted to a specific application, limiting their investigation to a rather simple code featuring two independent threads operating in lockstep, without diving into the intricacies required to support more complex general-purpose scenarios.

In contrast to these works, we presented a novel methodology enabling dual-issue execution of mixed integer and FP instruction sequences in an area- and energy-efficient processor, with virtually no area overhead due to our extensions. This is possible by leveraging Snitch's SSR and FREP extensions, which extend beyond the scope of dual-issue execution, enabling the core's high FPU utilization.

V. CONCLUSION

In this work, we presented COPIFT, a generic methodology to enable sustained dual-issue execution of mixed integer and floating-point instruction sequences in Snitch, a state-of-the-art open-source RISC-V processor. We further developed a set of ISA extensions to broaden the range of applications which can leverage our methodology. To evaluate the benefits of COPIFT-enabled dual-issue execution, we implemented optimized RV32G baseline codes for a variety of mixed integer and floating-point workloads on Snitch, and compared these to COPIFT-optimized variants leveraging our ISA extensions. We measured speedups of $1.47\times$ on average and a peak IPC of 1.75, proving COPIFT's effectiveness in enabling sustained dual-issue execution. We further compared the energy-efficiency of baseline and COPIFT-accelerated codes. Despite a small $1.07\times$ average increase in power consumption, COPIFT proves beneficial in overall energy terms, resulting in $1.37\times$ average energy savings on the measured benchmarks, and a peak $1.93\times$ saving on the `exp` kernel, proving that effective dual-issue execution is possible on area- and energy-efficient in-order cores.

ACKNOWLEDGMENT

This work has been supported in part by 'The European Pilot' project under grant agreement No 101034126 that receives funding from EuroHPC-JU as part of the EU Horizon 2020 research and innovation programme.

We would like to thank Lannan Jiang, Fabio Cappellini and Hakim Filali for their early experiments with Monte Carlo methods on Snitch.

REFERENCES

- [1] M. Horowitz, “1.1 computing’s energy problem (and what we can do about it),” in *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*, 2014, pp. 10–14.
- [2] J. Sevilla, L. Heim, A. Ho, T. Besiroglu, M. Hobbhahn, and P. Villalobos, “Compute trends across three eras of machine learning,” in *2022 International Joint Conference on Neural Networks (IJCNN)*, 2022, pp. 1–8.
- [3] J. Choquette, “Nvidia hopper h100 gpu: Scaling performance,” *IEEE Micro*, vol. 43, no. 3, pp. 9–17, 2023.
- [4] J. Burgess, “Rtx on—the nvidia turing gpu,” *IEEE Micro*, vol. 40, no. 2, pp. 36–44, 2020.
- [5] F. Zaruba, F. Schuiki, T. Hoefer, and L. Benini, “Snitch: A tiny pseudo dual-issue processor for area and energy efficient execution of floating-point intensive workloads,” *IEEE Transactions on Computers*, vol. 70, no. 11, pp. 1845–1860, 2021.
- [6] The risc-v instruction set manual volume i, unprivileged architecture, version 20240411. [Online]. Available: <https://github.com/riscv/riscv-isa-manual/releases/tag/20240411>
- [7] F. Schuiki, F. Zaruba, T. Hoefer, and L. Benini, “Stream semantic registers: A lightweight risc-v isa extension achieving full compute utilization in single-issue cores,” *IEEE Transactions on Computers*, vol. 70, no. 2, p. 212–227, Feb. 2021.
- [8] P. Scheffler, F. Zaruba, F. Schuiki, T. Hoefer, and L. Benini, “Sparse stream semantic registers: A lightweight isa extension accelerating general sparse linear algebra,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 12, p. 3147–3161, Oct. 2023.
- [9] E. Hammami and Y. Slama, “An overview on loop tiling techniques for code generation,” in *2017 IEEE/ACS 14th International Conference on Computer Systems and Applications (AICCSA)*, 2017, pp. 280–287.
- [10] A. Douillet and G. R. Gao, “Software-pipelining on multi-core architectures,” in *2007 16th International Conference on Parallel Architecture and Compilation Techniques (PACT)*, 2007, pp. 39–48.
- [11] *Monte Carlo Integration*. John Wiley & Sons, Ltd, 2014, ch. 2, pp. 31–64. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1002/9783527683147.ch2>
- [12] Gnu c library. [Online]. Available: <https://sourceware.org/git/?p=glibc.git>
- [13] J. R. Stevens, R. Venkatesan, S. Dai, B. Khailany, and A. Raghunathan, “Softtermx: Hardware/software co-design of an efficient softmax for transformers,” in *2021 58th ACM/IEEE Design Automation Conference (DAC)*, 2021, p. 469–474.
- [14] J. L. Hennessy and D. A. Patterson, *Computer Architecture, Fifth Edition: A Quantitative Approach*, 5th ed. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 2011.
- [15] H. B. Lozano and M. Ito, “Energy efficient dual-issue processor for embedded applications,” in *2015 IEEE 12th International Conference on Networking, Sensing and Control (ICNSC)*, 2015, pp. 544–549.
- [16] Y. Kra, Y. Shoshan, Y. Rudin, and A. Teman, “Hamsa-di: A low-power dual-issue risc-v core targeting energy-efficient embedded systems,” *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 71, no. 1, pp. 223–236, 2024.
- [17] K. Patsidis, D. Konstantinou, C. Nicopoulos, and G. Dimitrakopoulos, “A low-cost synthesizable risc-v dual-issue processor core leveraging the compressed instruction set extension,” *Microprocessors and Microsystems*, vol. 61, pp. 1–10, 2018.
- [18] M. Wyrzawski, P. Skrzypiec, and R. Szczygiel, “Hardware acceleration method using risc-v core with no isa extensions,” in *2024 31st International Conference on Mixed Design of Integrated Circuits and System (MIXDES)*, 2024, pp. 265–269.